Facultad de Ingeniería
Asignatura Arquitectura y diseño de software

Solución completa de la actividad: "Re-arquitectura de un sistema existente: de crecimiento acelerado a arquitectura resiliente"

1) Resumen ejecutivo

ParkKbus Suite nació como una solución rápida para un parqueadero pequeño y evolucionó hasta convertirse en una plataforma que hoy soporta múltiples sedes, usuarios, puntos de control e integraciones. Sin embargo, su arquitectura monolítica desplegada en una sola VM ha comenzado a mostrar límites severos de rendimiento, mantenibilidad, testabilidad y resiliencia. Las evidencias más fuertes son los tiempos de respuesta superiores a 8 segundos en horas pico, la fragilidad en despliegues, el bloqueo de la base de datos por reportes pesados y el impacto global cuando fallan integraciones externas.
A partir del análisis del caso, la decisión arquitectónica recomendada no es migrar de inmediato a microservicios puros, sino adoptar una estrategia híbrida y evolutiva:

- Monolito modular internamente para ordenar el dominio, reducir acoplamiento y mejorar testabilidad.
- Arquitectura Hexagonal + Onion en el core de negocio.
- CQRS parcial para separar escrituras críticas de lecturas pesadas.
- Extracción progresiva de capacidades de alto cambio o alto costo computacional, especialmente:
    - Reportes/analítica
    - Notificaciones
    - Integración con cámaras
- Mensajería asíncrona para desacoplar integraciones y proteger el core operacional.

Esta propuesta responde mejor a los RNF del enunciado: disponibilidad 99.5%, operaciones críticas menores a 2s, mantenibilidad, observabilidad, resiliencia y testabilidad, sin violar las restricciones de negocio ni exigir una migración traumática.

2) AS-IS: diagnóstico arquitectónico del sistema actual

2.1 Descripción de la arquitectura actual

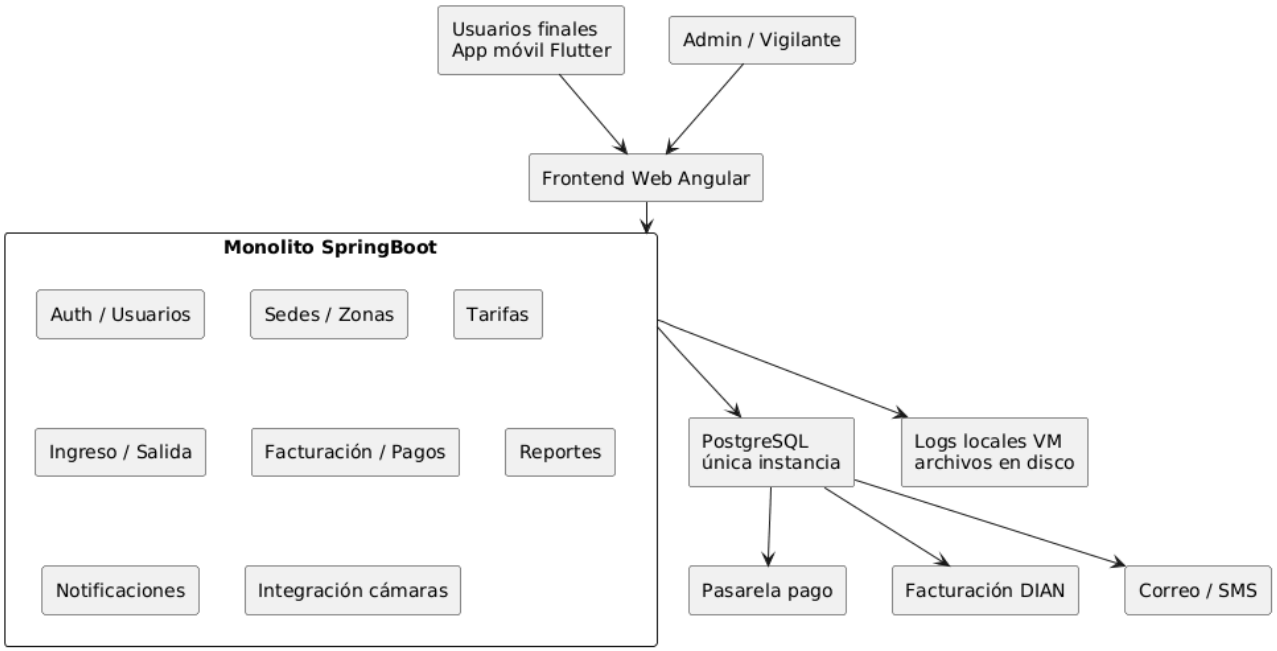
El sistema actual está implementado como un **monolito en Java Spring Boot** desplegado en un solo servidor virtual. Toda la lógica backend reside en un único proyecto. Se complementa con un frontend administrativo en Angular y una aplicación móvil en Flutter. La persistencia usa una sola instancia PostgreSQL. Las integraciones externas incluyen pasarela de pagos, facturación y notificaciones por correo/SMS. La reportería se realiza con consultas pesadas directas sobre la base de datos de operación y los logs se almacenan como archivos locales en el servidor.

Los módulos funcionales que hoy viven dentro del monolito son:

1. Usuarios y autenticación
2. Gestión de sedes y zonas
3. Tipos de vehículo y tarifas
4. Control de ingreso/salida
5. Facturación y pagos
6. Reportes y analítica
7. Notificaciones
8. Integración con cámaras

Lectura arquitectónica: Este diseño fue razonable al inicio porque permitió salir rápido al mercado. El problema no es “haber empezado con monolito”; el problema es **no haber evolucionado el diseño al mismo ritmo del crecimiento del negocio**.

2.2 Diagramas AS-IS



2.3 Cinco problemas críticos con análisis causa–efecto

Problema 1. Lentitud en horas pico
Síntoma: tiempos de respuesta superiores a 8 segundos en operaciones críticas.
Causa raíz: toda la carga de negocio, consultas, integraciones y reportes compite por los mismos recursos de cómputo y la misma base de datos.
Efecto: deterioro de la experiencia del vigilante/cajero, filas en los puntos de control, riesgo operativo y posible pérdida de ingresos.

Problema 2. Despliegue de alto riesgo
Síntoma: si falla una parte, cae todo.
Causa raíz: acoplamiento estructural dentro del monolito y empaquetado único.
Efecto: cualquier cambio pequeño exige desplegar toda la aplicación y aumenta el miedo al cambio del equipo.

Problema 3. Reportes bloquean la operación
Síntoma: reportería pesada afecta el resto del sistema.
Causa raíz: ausencia de separación entre carga transaccional y carga analítica; lecturas complejas se ejecutan contra la misma BD operacional.
Efecto: las operaciones de ingreso, salida y pago comparten contención con consultas de agregación y análisis.

Problema 4. Integraciones externas contaminan el core
Síntoma: fallos externos afectan todo el flujo.
Causa raíz: llamadas sin aislamiento suficiente a pasarela de pagos, facturación y notificaciones.
Efecto: una indisponibilidad externa se propaga al corazón del negocio y pone en riesgo ingreso/salida y pagos.

Problema 5. Baja testabilidad y alta fragilidad evolutiva
Síntoma: muchas dependencias acopladas, dificultad para probar.
Causa raíz: mezcla de lógica de dominio, acceso a datos, integración e infraestructura dentro de los mismos módulos o capas técnicas.
Efecto: pruebas lentas, baja confianza, alto costo de mantenimiento y menor velocidad de entrega.

2.4 Tres riesgos técnicos

Riesgo técnico 1. Punto único de falla: Al estar desplegado todo en una sola VM con una sola base operativa, la caída del nodo o degradación severa del recurso afecta el servicio completo.

Riesgo técnico 2. Deuda de acoplamiento: El crecimiento del sistema dentro del mismo proceso y del mismo esquema lógico aumenta la complejidad accidental y dificulta

refactorizar sin romper.

Riesgo técnico 3. Observabilidad insuficiente: Los logs locales en archivos impiden trazabilidad distribuida, análisis centralizado, correlación de errores y diagnóstico rápido. El RNF exige observabilidad y trazabilidad de transacciones, cosa que el AS-IS no cumple adecuadamente.

2.5 Tres riesgos de negocio

Riesgo de negocio 1. Pérdida de confianza operativa: Si el sistema se pone lento en ingresos/salidas, afecta directamente la operación de las sedes y la experiencia del cliente final.

Riesgo de negocio 2. Escalamiento costoso e ineficiente: Hoy escalar implica duplicar toda la VM, incluso si solo crece un módulo como reportes o notificaciones. Esto incrementa el costo sin precisión.

Riesgo de negocio 3. Menor capacidad de expansión: La organización quiere permitir nuevas integraciones, más sedes y resiliencia ante cambios tecnológicos. El AS-IS frena esa evolución.

2.6 Tres riesgos de negocio

Modularidad: Existe funcionalidad organizada por módulos de negocio, pero no verdadera independencia modular. Los módulos viven dentro del mismo despliegue y probablemente comparten demasiadas dependencias internas.

Separación de preocupaciones
No está bien resuelta. El sistema mezcla:

- operación transaccional,
- integraciones externas,
- reportería pesada,
- y observabilidad deficiente,
  dentro de una misma unidad de despliegue y sobre la misma base operativa.

Abstracción: Las integraciones con pagos, notificaciones y cámaras parecen estar demasiado embebidas. Falta una abstracción por puertos/adaptadores que aísle el dominio del detalle tecnológico.

Conclusión AS-IS: El problema principal no es solo “monolito vs microservicios”. El problema real es la ausencia de límites arquitectónicos claros, desacoplamiento e aislamiento operacional.

3) Comparación: monolito vs arquitectura distribuida

3.1 Ventajas del monolito en este caso

1. Simplicidad operativa inicial: un solo despliegue, una sola base, menos piezas.
2. Facilidad de desarrollo en etapa temprana: útil cuando el negocio era pequeño.
3. Menor sobrecarga de coordinación: ideal para equipos pequeños.
4. Transacciones simples: al estar todo en el mismo proceso y BD, ciertas operaciones son más directas.

3.2 Desventajas del monolito en este caso

1. Escalamiento grueso: no se puede escalar solo lo crítico.
2. Despliegue riesgoso: cambio pequeño, impacto grande.
3. Afectación cruzada: reportes y fallos externos afectan operaciones core.
4. Menor resiliencia: el acoplamiento aumenta el blast radius.
5. Complejidad evolutiva: el tamaño alcanzado lo hace frágil.

3.3 Ventajas de una arquitectura distribuida

1. Escalamiento selectivo por capacidades.
2. Aislamiento de fallos.
3. Despliegue independiente de piezas de alto cambio.
4. Mejor adecuación a cargas mixtas: operación vs analítica.
5. Mayor capacidad de evolución tecnológica.

3.4 Desventajas de una arquitectura distribuida

1. Mayor complejidad operativa.
2. Mayor complejidad en datos.
3. Mayor necesidad de observabilidad y automatización.
4. Más puntos de falla y latencia de red.
5. Costo de coordinación mayor, especialmente con equipo pequeño.

3.5 Decisión recomendada

Enfoque elegido: arquitectura híbrida
No propongo microservicios puros desde el día uno. Propongo:

- Monolito modular como base de reorganización interna
- Extracción selectiva de servicios donde exista mayor beneficio
- CQRS parcial
- Mensajería asíncrona
- PostgreSQL compartido durante transición, porque el enunciado obliga a mantenerlo inicialmente.

¿Por qué no microservicios puros?
Porque las restricciones del caso lo desaconsejan:

- máximo 2 desarrolladores adicionales,
- presupuesto limitado,
- no detener operación más de 2 horas,
- mantener misma BD en transición.

¿Por qué no quedarse igual?
Porque el AS-IS ya incumple o compromete los RNF de rendimiento, mantenibilidad, resiliencia, observabilidad y testabilidad.

Conclusión de decisión: La mejor solución arquitectónica no es la más "moderna" en abstracto, sino la que mejor balancea riesgo, costo, tiempo, restricciones y beneficio real.

4) Propuesta TO-BE

4.1 Estilo arquitectónico elegido

La propuesta combina:

1. **Arquitectura Hexagonal:** Para separar el dominio del detalle tecnológico. El dominio no depende de base de datos, APIs externas, colas o framework.

2. **Onion Architecture:** Para reforzar la dependencia hacia adentro:
   - centro: dominio
   - luego: aplicación
   - luego: infraestructura/adaptadores

3. **CQRS parcial:** No para todo el sistema, sino donde genera valor real:
   - escrituras críticas: ingreso, salida, pago, auditoría
   - lecturas intensivas: reportes, dashboards, analítica operativa

Esto responde directamente a la exigencia del taller de justificar estilos modernos y su alcance real.

4.2 Alcance real de adopción

Qué sí adopto ya

- Dominio desacoplado con puertos y adaptadores
- Casos de uso explícitos
- Separación comando/consulta en reportes y dashboards
- Extracción de integraciones no core

Qué no adopto de inmediato

- No separo cada módulo en microservicio independiente desde el inicio
- No hago base de datos por servicio desde fase 1
- No introduzco complejidad distribuida innecesaria en autenticación o tarifas si todavía no lo justifican

4.3 **Arquitectura objetivo**

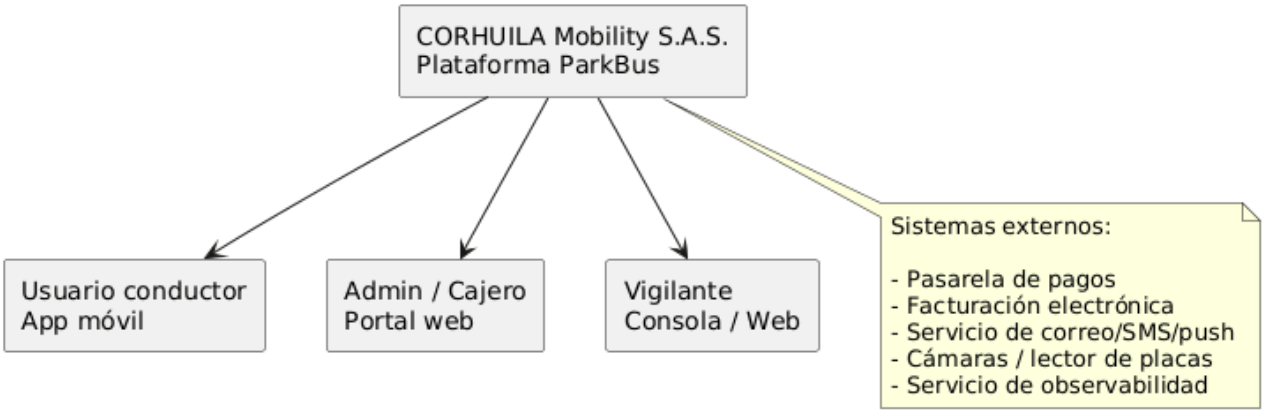
Capacidades principales
Core operacional

- Ingreso/Salida
- Tarifas
- Pagos
- Ticketing
- Cupos/Sedes/Zonas
- Usuarios/Roles
- Auditoría

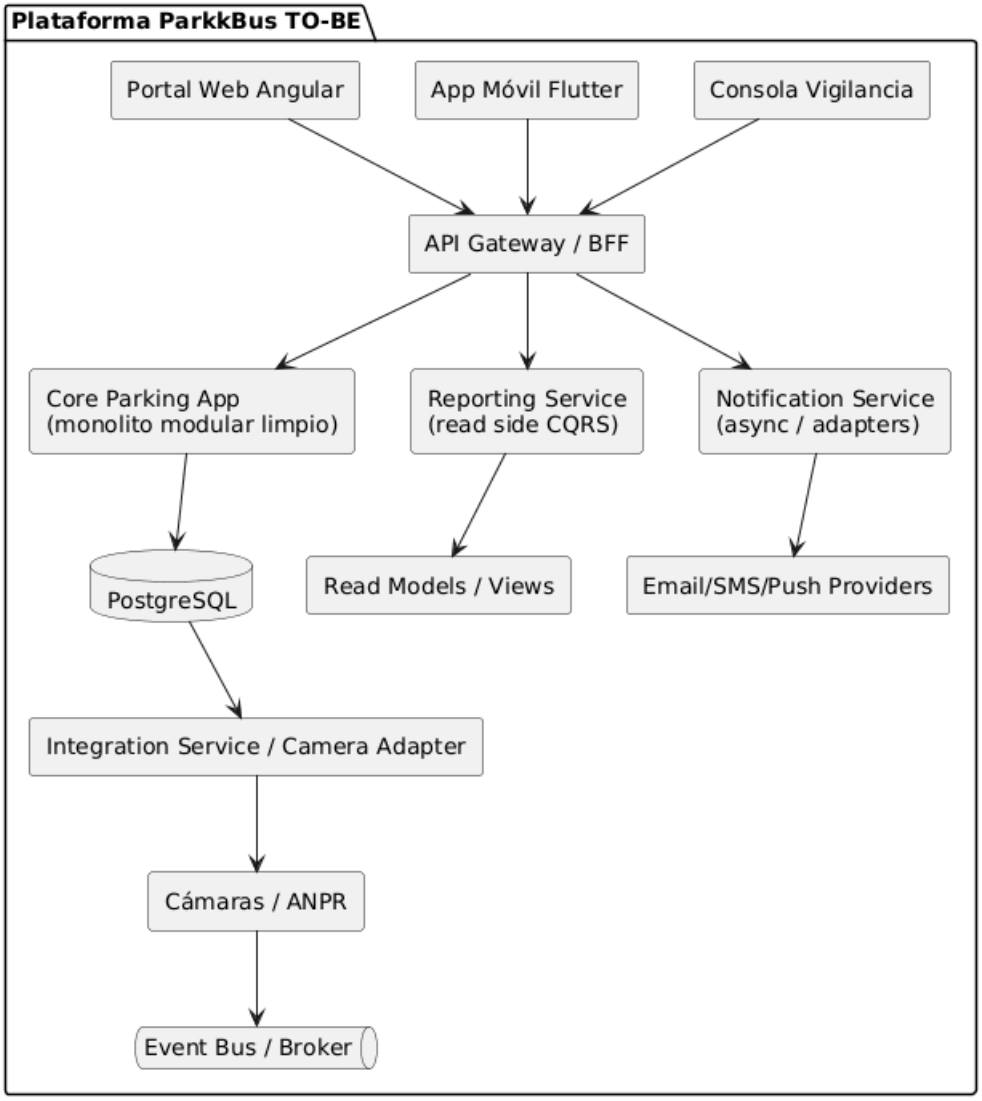
Capacidades desacopladas o extraíbles

- Notificaciones
- Integración con cámaras
- Reportes/analítica
- Integración con facturación externa

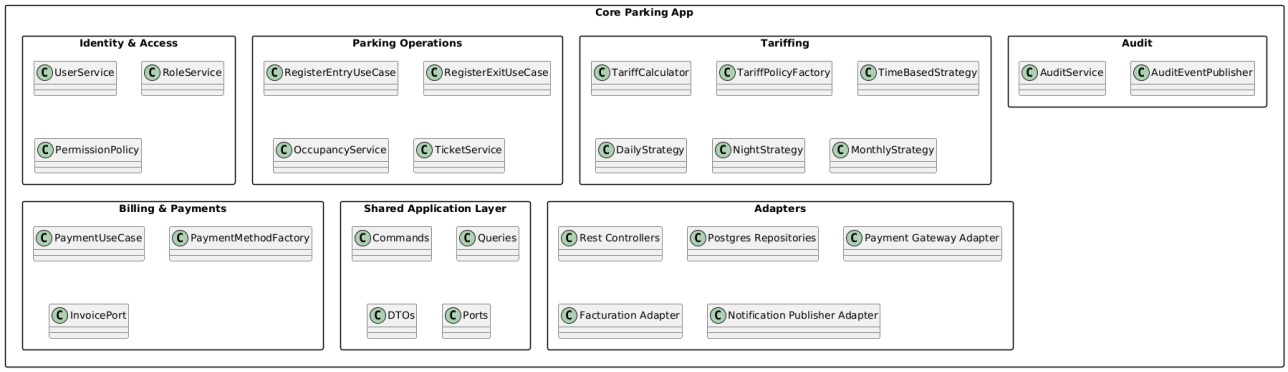
4.4 **Diagrama C1 – Contexto**



4.5 **Diagrama C2 – Contenedores**



4.6 **Diagrama C3 – Componentes del Core Parking App**



5) Patrones de diseño aplicados

El taller exige mínimo 2 creacionales, 2 estructurales y 2 de comportamiento, con uso concreto.

5.1 Patrones creacionales

1. Factory Method
Uso: creación de estrategia de cálculo de tarifa según modalidad.
Ejemplo:

- Tarifa por hora
- Tarifa por día
- Tarifa nocturna
- Tarifa mensual

Ventaja: evita if/else gigantes y centraliza la selección de la política tarifaria.

2. Abstract Factory
Uso: construcción de adaptadores por proveedor externo:

- proveedor de notificaciones,
- proveedor de facturación,
- proveedor de pagos.

Ventaja: permite cambiar proveedor sin contaminar el dominio.

5.2 Patrones estructurales

3. Adapter
Uso: envolver APIs externas:

- pasarela de pagos,
- servicio de facturación,
- servicio SMS/correo,
- integración con cámaras.

Ventaja: el dominio conoce interfaces propias, no SDKs externos.

4. Decorator
Uso: enriquecer servicios con capacidades transversales:

- logging,
- métricas,
- auditoría,
- retry controlado.

Ventaja: evita ensuciar la lógica central con preocupaciones transversales.

5.3 Patrones de comportamiento

5. Strategy
Uso: cálculo de tarifas.
Cada modalidad implementa una política distinta.

6. Observer / Publish-Subscribe
Uso: ante eventos del dominio:

- vehículo ingresó,
- vehículo salió,
- pago completado,
- anomalía detectada.

Ventaja: el core publica eventos y otros componentes reaccionan sin acoplamiento directo.

7. Command
Uso adicional valioso: representar casos de uso de escritura:

- RegisterVehicleEntryCommand
- RegisterVehicleExitCommand
- ProcessPaymentCommand

Ventaja: hace más claro CQRS y permite trazabilidad.

6) Decisiones arquitectónicas clave

Decisión 1. Mantener PostgreSQL durante la transición

Alternativa descartada: dividir de inmediato en bases por servicio.
Justificación: el caso obliga a mantener el mismo PostgreSQL al menos durante la transición. Además, una división prematura de datos aumentaría demasiado el riesgo.
Impacto esperado: menor riesgo migratorio, menor costo inicial, transición más realista.

Decisión 2. Adoptar arquitectura híbrida y no microservicios puros

Alternativa descartada: migración completa a microservicios.
Justificación: equipo y presupuesto limitados.
Impacto: equilibrio entre mejora arquitectónica y capacidad real de ejecución.

Decisión 3. Separar reportes del flujo transaccional mediante CQRS parcial
Alternativa descartada: seguir consultando la BD operacional para analítica.
Impacto: mejor tiempo de respuesta en operaciones críticas y menor contención.

Decisión 4. Desacoplar integraciones con mensajería/eventos
Alternativa descartada: invocaciones síncronas directas desde el core en todos los casos.
Impacto: mayor resiliencia y menor propagación de fallos.

Decisión 5. Logs centralizados y trazabilidad

Alternativa descartada: continuar con archivos locales.
Impacto: mejora soporte, monitoreo y cumplimiento de observabilidad.

7) Stack tecnológico propuesto

El taller pide justificar backend, frontend, BD, mensajería, despliegue y herramientas.

7.1 Backend

Opción propuesta: Java Spring Boot
No recomiendo reescribir el backend a otro lenguaje en esta fase.

Justificación arquitectónica:
- Reduce riesgo de transición.
- Reutiliza experiencia del equipo y activos existentes.
- Spring Boot permite implementar perfectamente Hexagonal, Onion y CQRS.
- Minimiza costo de reaprendizaje.

Conclusión:
La re-arquitectura no requiere cambiar de lenguaje; requiere **cambiar diseño**.

7.2 Frontend web

Angular:
Se mantiene.

Justificación:
- Ya existe.
- Permite modularizar por dominio.
- Evita costo adicional de reescritura.

7.3 App móvil

Flutter
Se mantiene.

Justificación:
- Ya existe.
- No es la fuente principal del problema arquitectónico actual.

7.4 Base de datos

PostgreSQL
Se mantiene como base principal en la transición.

Complementos recomendados:
- vistas materializadas o read models
- réplicas de lectura en una fase posterior si el presupuesto lo permite
- particionado por fecha/sede en tablas críticas si el volumen lo exige

7.5 **Mensajería / integración**

RabbitMQ o Kafka
Para este caso, por equilibrio costo/complejidad, propondría **RabbitMQ** inicialmente.

Por qué RabbitMQ primero:
- Menor complejidad operativa para equipo pequeño
- Muy útil para colas de integración y eventos operativos
- Suficiente para notificaciones, integración de cámaras y desacople transaccional ligero

Dónde usarlo:
- eventos de pago completado
- solicitud de notificación
- integración de placas/cámaras
- generación diferida de reportes o proyecciones

7.6 **Despliegue**

Contenedores sobre VM
No propongo saltar de una vez a Kubernetes.

Fase recomendada:
- Docker
- Docker Compose o equivalente
- 2 a 3 VMs según ambiente

Justificación:
- mejor aislamiento que una sola VM monolítica
- menor complejidad que orquestadores pesados
- adecuado al tamaño del equipo

7.7 **Observabilidad**

- logs centralizados: ELK u OpenSearch + dashboards
- métricas: Prometheus + Grafana
- trazabilidad: OpenTelemetry

7.7 **Herramientas de modelado**

- PlantUML para versionar diagramas
- draw.io para visualización académica
- ADRs en Markdown dentro del repositorio

7.8 **Modelo de datos conceptual**

Entidades clave
- Usuario
- Rol
- Permiso
- Sede
- Zona
- Cupo

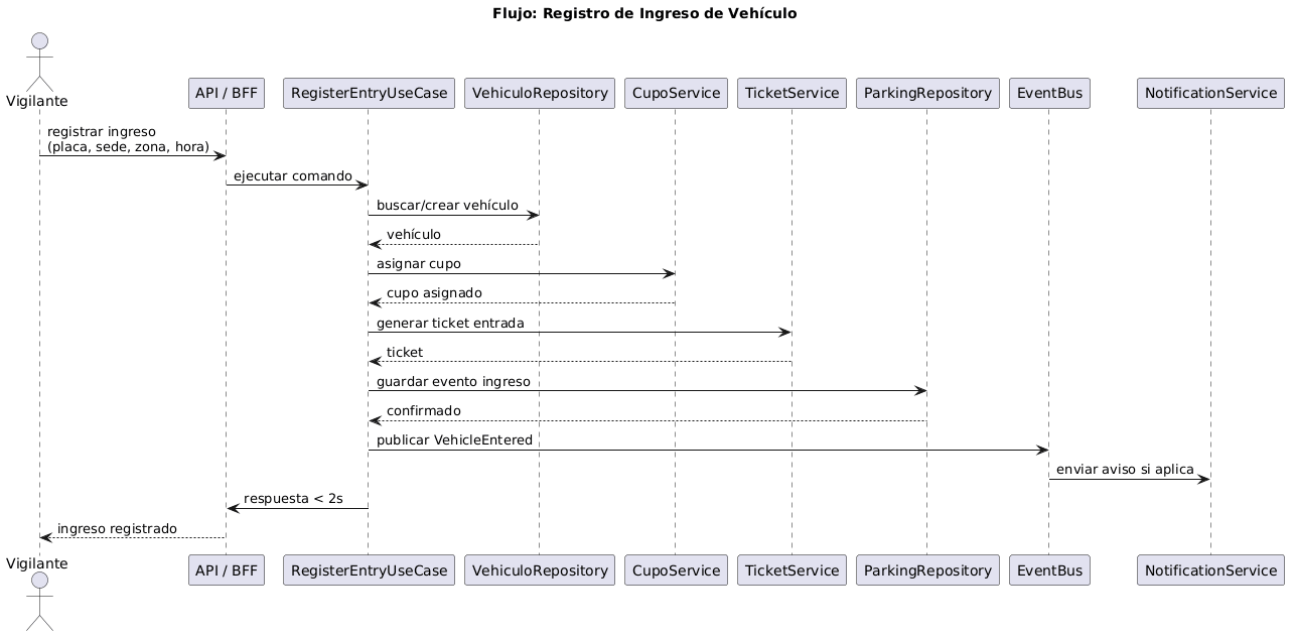
- TipoVehiculo
- Tarifa
- Vehiculo
- IngresoSalida
- Ticket
- Pago
- Factura
- Notificacion
- EventoCamara
- Auditoria

Relaciones principales

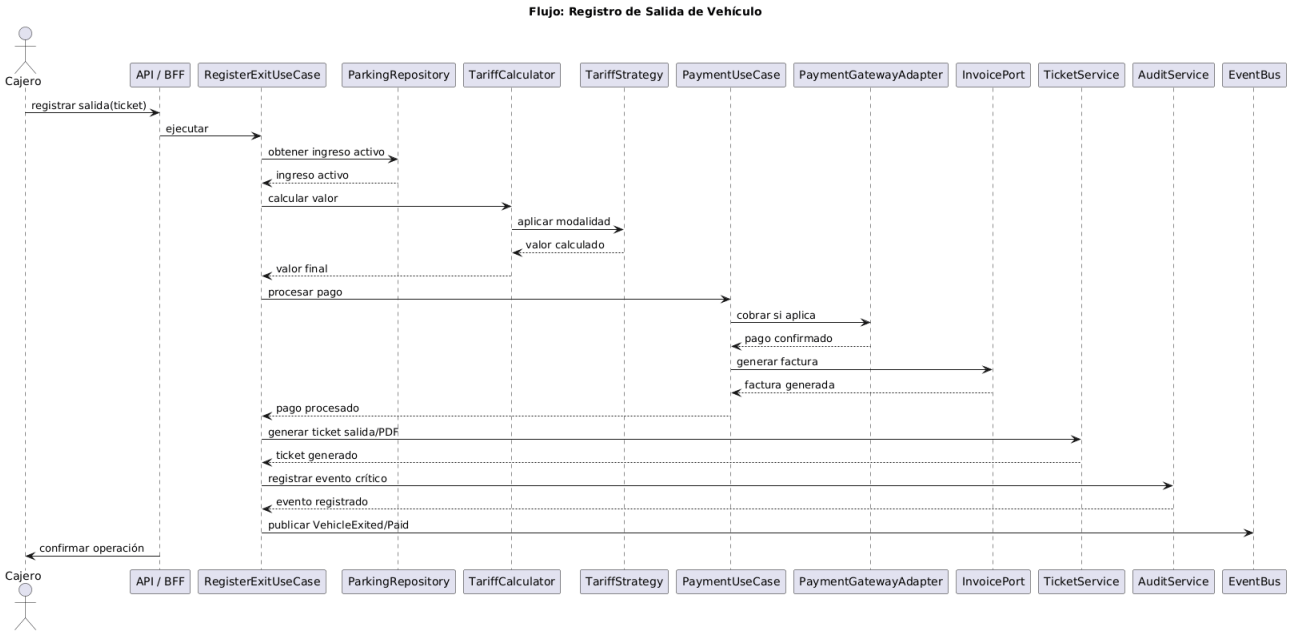
- Una **Sede** tiene muchas **Zonas**
- Una **Zona** tiene muchos **Cupos**
- Un **TipoVehiculo** se asocia a muchas **Tarifas**
- Un **Vehiculo** registra muchos **IngresoSalida**
- Un **IngresoSalida** puede generar uno o más **Pagos**
- Un **Pago** puede originar una **Factura**
- Un **Usuario** ejecuta acciones auditables sobre distintas entidades
- Un **EventoCamara** puede asociarse a un **Vehiculo** o a una detección pendiente

9) Diagramas de secuencia

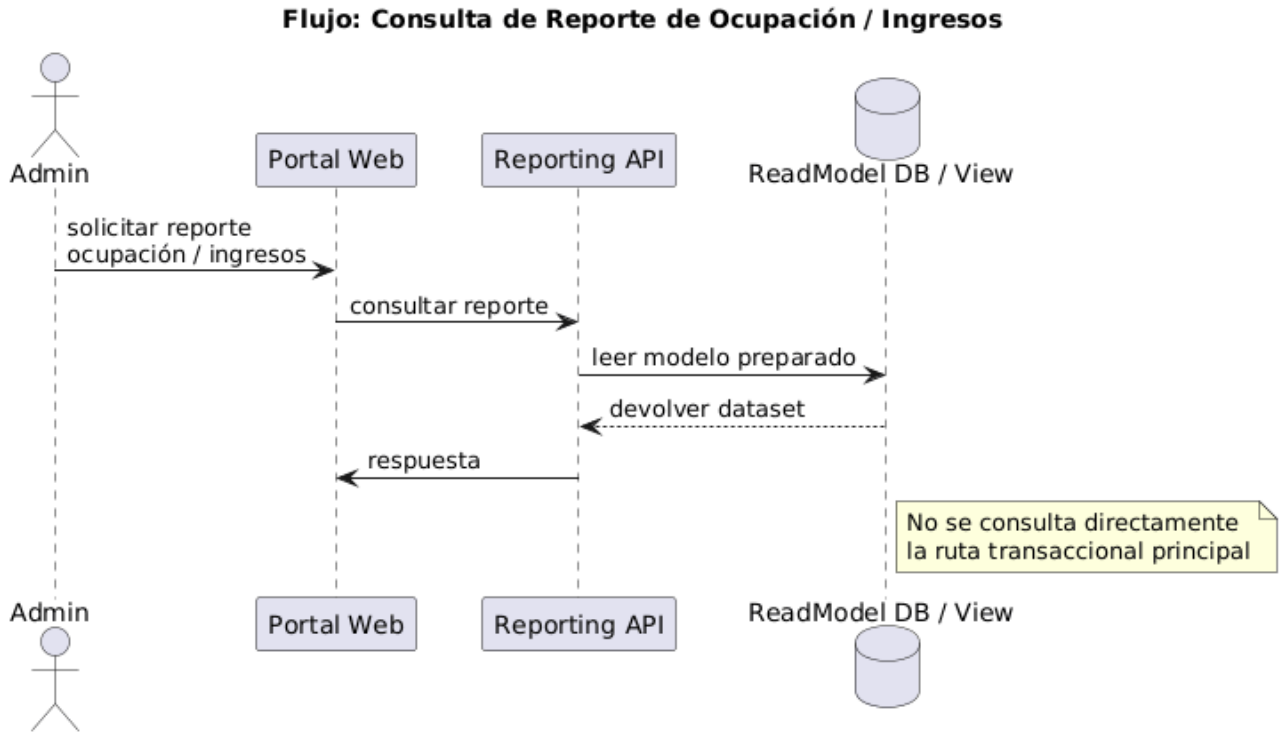
9.1 **Ingreso de vehículo**



9.2 **Salida + cálculo + pago**



9.3 Generación de reporte sin afectar operaciones



10) Plan de migración

El enunciado pide mínimo 3 fases, sin apagar operación, con validación y mitigación.

Estrategia general
Enfoque: Strangler Fig Pattern
No reemplazar todo de una vez.
Se rodea el sistema actual, se reorganiza internamente y luego se extraen capacidades específicas.

Fase 1. Ordenamiento interno del monolito

Objetivo: convertir el monolito en un monolito modular limpio.

Qué se hace
- separar dominio, aplicación e infraestructura
- introducir puertos/adaptadores
- encapsular integraciones externas
- identificar bounded contexts funcionales
- centralizar logs y métricas
- definir comandos y consultas explícitas

Por qué primero
Porque sin esta limpieza, cualquier extracción posterior sería caótica.

Riesgos
- refactorización interna con regresiones

Mitigación
- pruebas de regresión
- feature toggles
- despliegues pequeños

Fase 2. Separación de lecturas pesadas y observabilidad
Objetivo: proteger las operaciones críticas.

Qué se migra
- reportes y analítica
- dashboards de ocupación e ingresos
- consultas administrativas pesadas

Cómo
- CQRS parcial
- vistas materializadas / tablas de proyección
- procesos de actualización asíncronos o por lotes

Validación
- medir caída del tiempo de respuesta en ingreso/salida/pago
- comparar exactitud entre reporte viejo y reporte nuevo

Fase 3. Desacoplar notificaciones e integraciones externas

Objetivo: evitar que fallos externos tumben el core.

Qué se migra
- correo/SMS/push
- eventos de cámaras
- facturación no bloqueante donde sea posible

Cómo
- broker de mensajes
- colas de retry
- circuit breaker
- timeouts controlados

Validación
- pruebas de caída simulada de proveedores
- verificar que ingreso/salida siga operando

Fase 4. Extracción selectiva de servicios

Objetivo: convertir capacidades de alta variabilidad en servicios separados.
Candidatos
- Reporting Service
- Notification Service
- Camera Integration Service

Condición
Solo después de estabilizar el monolito modular.

Cómo evitar interrupciones
- despliegue blue/green o rolling sencillo
- feature flags
- endpoints paralelos temporales
- migración por sede piloto
- ventanas de cambio fuera de picos

11) Presupuesto estimado

El taller pide una estimación coherente y defendible, no exacta.

11.1 Infraestructura mensual estimada

Escenario propuesto
- 2 VMs de aplicación
- 1 VM o servicio administrado para broker
- PostgreSQL mantenido
- almacenamiento de logs/monitoreo

Estimación mensual referencial
- Cómputo aplicaciones: USD 250–400
- BD PostgreSQL: USD 180–300
- Broker / integración: USD 60–120
- Logs/monitoreo/almacenamiento: USD 80–150
- Backups / almacenamiento adicional: USD 40–80

Total mensual estimado

USD 610–1.050
Esta cifra es razonable para una solución de transición seria sin irse a una plataforma cloud sobredimensionada.

11.2 Costos de personal

La restricción dice que no pueden contratar más de dos desarrolladores adicionales.
Equipo sugerido

- 1 arquitecto/líder técnico
- 2 desarrolladores backend
- 1 desarrollador frontend
- 1 QA compartido
- 1 DevOps parcial/compartido

Refuerzo adicional permitido

- hasta 2 desarrolladores adicionales temporales

Tiempo estimado

- Fase 1: 6–8 semanas
- Fase 2: 4–6 semanas
- Fase 3: 4–6 semanas
- Fase 4: 6–8 semanas

Duración total
20–28 semanas

12) Conclusiones

La solución recomendada para ParkKbus Suite no consiste en “romper el monolito” por moda, sino en evolucionarlo con criterio arquitectónico. El análisis del caso muestra que el sistema actual falla principalmente por acoplamiento excesivo, competencia entre cargas transaccionales y analíticas, dependencia directa de integraciones externas y baja testabilidad. Estos problemas son incompatibles con los RNF del negocio, especialmente disponibilidad, rendimiento, mantenibilidad, observabilidad y resiliencia.
Por eso, la mejor decisión es una arquitectura híbrida evolutiva:

- monolito modular limpio,
- Hexagonal + Onion en el core,
- CQRS parcial,
- eventos y mensajería para desacoplar,
- extracción progresiva de servicios críticos.

Esta propuesta es más defendible que microservicios puros porque respeta las restricciones reales del caso: poco margen de parada, presupuesto limitado, equipo pequeño y continuidad de PostgreSQL durante la transición.
Como solución académica y profesional, esta respuesta demuestra que la arquitectura correcta no es la más compleja, sino la que maximiza valor, reduce riesgo y permite evolucionar con control.